



THE UNIVERSITY
of EDINBURGH

Computational Fluid Dynamics Exercise

Serial Performance

Practical Introduction to HPC



Contents

1	Introduction and Aims.....	3
2	Exercise	3
2.1	Download, transfer and compile	3
2.2	Run the program	3
2.3	Viewing the output	5
3	Quantifying the serial performance	6
4	Extra Exercise	7

1 Introduction and Aims

This exercise takes an example from one of the most common applications of HPC resources: Fluid Dynamics. You should be familiar with the problem as it was introduced in one of the lectures. The aim of this exercise is to quantify the performance of the serial code run on one core and to investigate how it changes with increasing problem size. This exercise should give you an insight into how access to various levels of memory affects performance on modern processors.

2 Exercise

2.1 Download, transfer and compile

Download the file `cfid_serial.tar.gz` from the *Learn* page and copy it to your `/work` directory on ARCHER2 (using the **scp** command) as you did for the previous exercise. Unpack using the **tar** command:

```
UserID@ln01:~> tar -zxvf cfid_serial.tar.gz
```

There are 3 directories with serial C, Fortran and Python code. In this exercise, you will be using either the C or Fortran versions. To compile, go to the **cfid/C-SER** or **cfid/F-SER** directory and type **make** to create the **cfid** executable.

Note: If you have cloned the Gitlab repository, you will already have this code in this week's exercise directory. It will have the same directory structure described above.

2.2 Run the program

Use your preferred editor to look at the `cfid.slurm` script – you will see it's almost identical to the serial scripts for the image sharpening exercise. The only difference is the executable and its arguments:

```
# Launch the serial job using 1 thread
srun ./cfid 4 5000
```

The arguments above have the following meaning:

- **./cfid** - use the `cfid` executable located in the current directory
- **4** - sets the scale factor, which controls the size of the problem, to 4
- **5000** - sets the number of iterations to 5000

The minimum problem size (scale factor = 1) is taken as a 36x36 grid. The actual problem size can be chosen by scaling this basic size; for example, setting the scale factor to 4 means using a 144x144 grid ($36 \times 4 = 144$).

The number of iterations is not particularly important, as we are interested in the time per iteration. You can increase the number of iterations to ensure that the code does not run too fast for the smaller system sizes and decrease the number of iterations for larger problem sizes to reduce the run time.

Try to set number of iterations so that the run takes at least several or tens of seconds. If you decided to do longer runs, remember to adjust the walltime requested in the script.

Once you have run the job via the *Slurm* batch system, the output file should look something like this (depending on exact parameters):

```
Scale Factor = 4, iterations = 5000
Irrotational flow
Running CFD on 144 x 144 grid in serial

Starting main loop...

Completed iteration 1000
Completed iteration 2000
Completed iteration 3000
Completed iteration 4000
Completed iteration 5000

... finished
After 5000 iterations, the error is 0.000216569
Time for 5000 iterations was 0.088907 seconds
Each iteration took 1.77814e-05 seconds

Writing data files ...
... done!

Written gnuplot script 'cfd.plt'
... finished
```

Run the code for different system sizes and fill in Table 1. You can run multiple jobs inside a single Slurm script – these will be run sequentially one after another using the same resource allocation. To do that you can simply have multiple `srun` commands, e.g.:

```
srun ./cfd 1 5000
srun ./cfd 4 5000
```

Alternatively, you can make use of a bash loop – specifying the range of values (e.g. 1, 4 and 8) for a variable (*scale_factor*) and substituting a different value at each iteration e.g.:

```
for scale_factor in 1 4 8
do
    srun ./cfd $scale_factor 5000
done
```

2.3 Viewing the output

The code also produces some graphical output in the form of a *gnuplot* file *cfid.plt*, which when plotted looks similar to Figure 1. You can view this using the following command:

```
UserID@ln01:~> gnuplot -persist cfd.plt
```

Note: **remember to load a gnuplot module** with the command `module load gnuplot`! Also, for smaller number of iterations, the output image may look a bit strange. This is because the calculation may have ended before the solution has been reached.

If the fluid is flowing down the right-hand edge then along the bottom (see Figure 2), rather than through the middle of the cavity as in Figure 1, then this is an indication that the solution algorithm has not yet converged. Convergence requires more iterations on larger problem sizes. This is not a problem in terms of performance figures, but it is worth running with more iterations just to check that the code is functioning correctly.

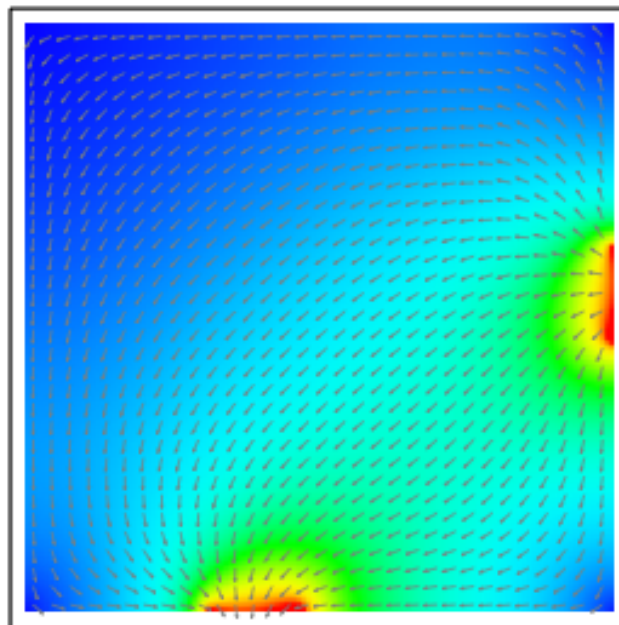


Figure 1: Output Image for the converged irrotational flow CFD simulation.

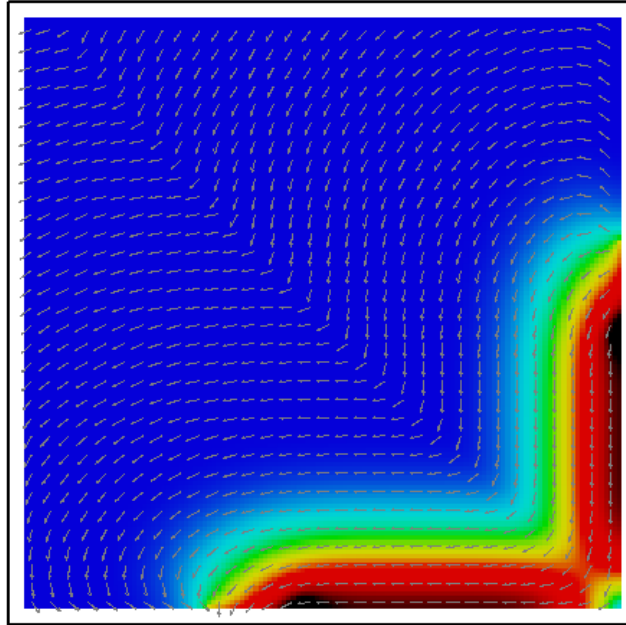


Figure 2: Output image for not converged irrotational flow CFD simulation.

3 Quantifying the serial performance

To quantify the performance, you need to calculate the number of updated cells per second. To do this you multiply the size of the grid by the number of iterations and divide by the execution time.

System Size	# Iterations	Execution Time (sec)	Updated cells/sec
36			
72			
144			
288			
576			
1152			
2304			
4608			

Table 1

For example, using the example data from page 4:

$$\text{updated cells/sec} = \frac{144 * 144 * 5000}{0.204869 \text{ sec}} \approx 506079495$$

Plot the number of updated cells/sec versus the system size (or scale factor) and try to answer the following questions:

- How does the number of updated cells change with increasing system size?
- Can you explain the observed behaviour? Is it something you had expected?
- Knowing that two system-size arrays are needed to perform the calculation estimate how much memory each system size requires.
- How do you think it is related to the hardware architecture? Compare those sizes with the sizes of each cache level. What do you see?

4 Extra Exercises

4.1 Reynolds number

The simulations you had done are not very realistic as they assume the fluid has no viscosity. However, the code can actually simulate fluids with a finite viscosity as well. You can pass a third parameter to the program, the Reynolds Number Re , which here is effectively the input velocity of the fluid.

For example, setting $Re = 1.8$ (`./cfd 4 5000 1.8`) will produce the image shown in Figure 3. Note the existence of whirlpools in the upper and lower right corners, which is much more realistic for a real fluid.

Simulating the whirlpools increases the computation required so the code will be slower, although the actual amount of computation is independent of the particular value chosen for Re .

Note: the code becomes unstable for values of $Re > 3.7$, and setting Re to zero is not the same as not specifying it (i.e. more calculations are still performed but with Re value set to zero).

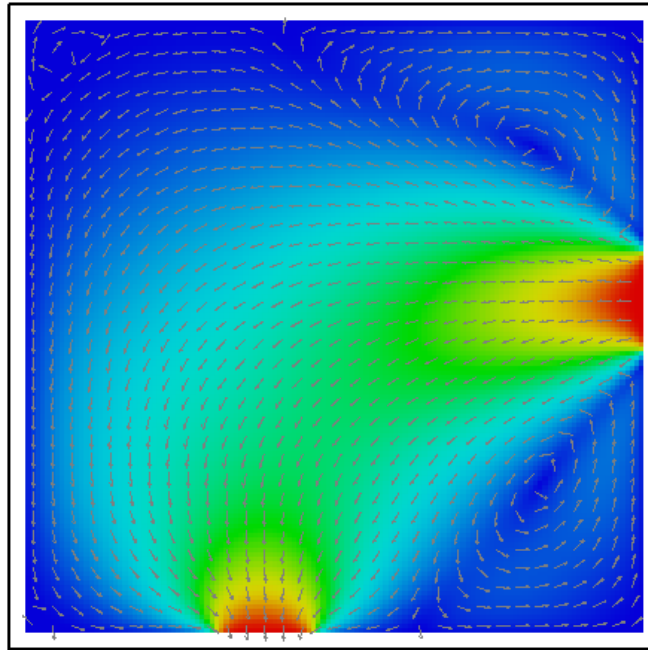


Figure 3: Simulation with $Re = 1.8$.

Pick a value for Re and repeat the previous steps to fill table 2. Then plot the number of updated cells/sec.

System Size	# Iterations	Execution Time (sec)	Updated cells/sec
36			
72			
144			
288			
576			
1152			
2304			
4608			

Table 2

Compare the performance graphs you have produced with and without Reynolds number.

- How do they differ?
- Is it something you were expecting?
- Can you explain the difference?

Looking at the source code should give you an idea of how much additional work is required when the effects of viscosity are included.

4.2 Estimating FLOPS

One common way of comparing the computational performance of different hardware (such as CPUs or GPUs) is to compute the number of floating point operations that can be performed every second, abbreviated as FLOPS (Floating point Operations per Second). Because of the speed of modern hardware this is often expressed in giga-FLOPS (GFLOPS) or tera-FLOPS (TFLOPS). This can be computed directly based on the clock speed of your processor.

$$FLOPS = \text{cores} \times \frac{\text{cycles}}{\text{second}} \times \frac{\text{floating point operations}}{\text{cycle}}$$

For example, the AMD EPYC 7742 processors in ARCHER2 can perform a maximum of 16 floating point operations per cycle. They operate at 2.25 GHz (cycles/second) and have 64 cores. Therefore, the maximum expected FLOPS they can achieve is:

$$64 \times 2.25 \times 10^9 \times 16 = 2.304 \text{ TFLOPS}$$

Based on your calculations of the number of cells updated per second it is possible to estimate the FLOPS achieved by this program. The loop that performs most of the work in the code is in `jacobistep` summarised below in pseudocode:

```
for(i=1;i<=n_rows;i++)
{
    for(j=1;j<=n_cols;j++)
    {
        psinew[i][j]=0.25*(psi[i-1][j]+psi[i+1][j]+psi[i][j-1]+psi[i][j+1]);
    }
}
```

From this formula compute the number of floating point operations that are performed per cell update and use the data you collected above to estimate the number of FLOPS achieved by this program. How does this compare with the theoretical maximum? What factors might lead to discrepancies between the value you calculated and the theoretical maximum? Do you think this is a good way of evaluating the performance of a program?